CURRENT WORKING SYSTEM ARCHITECTURE

Component Topology, State Management & Server-Side AI Gateway

ARCHITECTURAL BLUEPRINT (EXISTING CODEBASE) REACT 19 • TYPESCRIPT • NODE.JS PROXY

1. PRESENTATION LAYER (DUAL PORTALS)

Customer Portal Widget
src/components/CustomerPortal.tsx
Customer submits order inquiry (SD-1001 / WL-2001); views verified, agent-approved replies in live thread.

Agent Copilot Workstation
src/components/AgentPortal.tsx
Live ticket queue, order context verification, cited policy cards, and 1-click Approve & Send draft composer.

▼ (Reactive Store & Deterministic State)

2. CLIENT RUNTIME & GROUNDED RULE ENGINE

Centralized State Store
src/services/store.ts
In-memory reactive pub/sub store with optimistic ticket updates and cross-portal synchronisation.

Deterministic Policy Retrieval
src/services/retrieval.ts
Computes delivery date deltas against brand windows (SD: 7 days vs WL: 30 days) and flags invalid reference IDs.

▼ (POST /api/gemini with Grounded Context)

3. SECURE SERVER-SIDE AI GATEWAY

Serverless AI Proxy
api/gemini.js & server.cjs
Isolates GEMINI_API_KEY on the Node.js server. Zero client credential exposure to the browser.

Model & Fallback Engine
src/services/aiGenerator.ts
Primary: Google Gemini 1.5 Flash. Local deterministic generator engages automatically if offline.

▼ (Human Agent Approval & Audit Logging)

4. AUDIT TRAIL & DATABASE PREPARATION

Audit Log & Multi-Tenant Migrations
src/components/AuditLogView.tsx & supabase/migrations/
In-memory audit log recording model outputs, confidence scores, and agent revisions. Prepared PostgreSQL schemas with Row-Level Security.

EXISTING CODEBASE VERIFICATION

Component	Implementation Path	Verification Status
Frontend & UI	`src/components/`	React 19 + TypeScript + Vite. Fully functional dual-portal UI.
Deterministic Rules	`src/services/retrieval.ts`	23/23 Vitest acceptance tests passing.
Server AI Proxy	`api/gemini.js`, `server.cjs`	Zero token leakage. Verified with Gemini 1.5 Flash.
Database Schema	`supabase/migrations/`	10 tables with PostgreSQL Row-Level Security policies.